



المدرسة الوطنية للعلوم التطبيقية - بني ملال
ⵜⴰⴷⵓⴷⴰ ⵜⴰⴳⴷⵓⴷⴰ ⵜⴰⴳⴷⵓⴷⴰ ⵜⴰⴳⴷⵓⴷⴰ - ⵎⵉⵎⵉⵍⵍⴰⵍ
Ecole Nationale des Sciences Appliquées - Béni Mellal

Module :

Génie Logiciel

Compte Rendu du TP 2 :
GIT

2^{ème} année : Filière Transformation Digitale Industrielle

Réalisé par :

ECH-CHOKHMANY Loubna

Professeur :

- Pr. BE.ELBAGHAZAOU

Année universitaire 2025 / 2026

I. Introduction :

Ce rapport présente les travaux pratiques réalisés sur Git et Node.js. L'objectif principal est de maîtriser les commandes fondamentales de Git : Créer et gérer un projet Node.js, manipuler les branches, effectuer des fusions (merge), résoudre des conflits, utiliser rebase et cherry-pick, collaborer avec GitHub.

Le projet consiste en une application Node.js simple (mon-projet-node) qui sert de support pour illustrer les différentes fonctionnalités de Git.

II. Étape 1 - Initialisation du projet Node.js :

La phase d'initialisation est cruciale pour établir un environnement de développement structuré, reproductible et conforme aux standards de l'industrie.

1. Configuration du répertoire de travail :

Le projet est amorcé par la création d'un dossier dédié et l'activation des outils de gestion de code et de dépendances:

- **Création du dossier (`mkdir mon-projet-node`)** : Création du répertoire racine destiné à accueillir l'ensemble des ressources du projet.
- **Initialisation Git (`git init`)** : Transforme le répertoire local en un dépôt Git, permettant de suivre l'historique complet des modifications et de gérer les versions du code.
- **Initialisation Node (`npm init -y`)** : Génère automatiquement le fichier `package.json`, qui sert de descripteur de projet pour répertorier les métadonnées et les dépendances nécessaires à l'application.

2. Gestion de la documentation et de l'environnement :

Deux fichiers fondamentaux sont créés pour garantir la clarté et la propreté du dépôt:

- **Le fichier `README.md`** : Constitue le manuel d'instruction indispensable ; il présente les objectifs du projet, les étapes d'installation et les guides d'exécution du code.

- **Le fichier `.gitignore`** : Son rôle est d'exclure du suivi Git les fichiers inutiles ou volumineux, tels que le répertoire `node_modules/`, afin de maintenir un dépôt léger et sécurisé.

3. Mise en place du point d'entrée applicatif :

La création d'un fichier minimal permet de valider la configuration technique avant d'entamer le développement de la logique métier:

- **Fichier `index.js`** : Défini comme le point d'entrée principal de l'application, il contient la logique du code et permet l'exécution du programme via l'interpréteur Node.js.
- **Premier Commit** : L'ajout (`git add`) et la validation (`git commit`) des fichiers initiaux créent un instantané (snapshot) de référence, marquant le début officiel du cycle de vie du logiciel.

Exécution : `npm run start` → Hello Node

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node
$ npm run start

> mon-projet-node@1.0.0 start
> node index.js

Hello Node
```

III. Étape 2 - Gestion des branches et développement modulaire :

L'objectif de cette phase est d'expérimenter le flux de travail collaboratif en isolant le développement de nouvelles fonctionnalités du tronc principal (main).

1. Isolation des développements (Branchement) :

L'utilisation des branches permet de travailler sur des fonctionnalités spécifiques sans compromettre la stabilité du code principal:

- **Création de la branche (`git branch dev`)** : Initialise une branche nommée `dev` qui servira de bac à sable pour les nouveaux développements.

- **Changement de contexte** (`git checkout dev`) : Déplace le pointeur de travail sur la branche `dev`, garantissant que toutes les modifications ultérieures seront isolées de la branche `main`.

2. Implémentation de la logique métier :

Une fois l'environnement isolé, nous procédons à l'enrichissement fonctionnel du fichier `index.js`:

- **Fonction** `addition(a, b)` : Implémentation d'une fonction mathématique simple retournant la somme de deux paramètres.
- **Modularité** (`module.exports`) : Exportation de la fonction pour permettre sa réutilisation par d'autres modules du projet.
- **Test d'exécution** (`console.log`) : Vérification du bon fonctionnement de la logique avec des valeurs de test (ex: addition de 5 et 3).

3. Archivage des modifications :

Après avoir validé le fonctionnement du code, les changements sont enregistrés dans l'historique de la branche `dev`:

- **Validation des changements** (`git commit -m "..."`) : Enregistre un message clair décrivant l'ajout de la fonction `addition`, facilitant la traçabilité pour les futurs collaborateurs.

Création branche dev et commit de la fonction addition

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (master)
$ git branch dev

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (master)
$ git checkout dev
M   package.json
Switched to branch 'dev'

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (dev)
$ notepad index.js

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (dev)
$ git add index.js
warning: in the working copy of 'index.js', LF will be replaced by CRLF the next time Git touches it
```

Résultat du commit sur la branche dev

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (dev)
$ git commit -m "Ajout de la fonction addition dans index.js"
[dev bac425a] Ajout de la fonction addition dans index.js
1 file changed, 9 insertions(+), 1 deletion(-)
```

IV. Étape 3 - Fusion des développements (Merge) :

Cette étape consiste à intégrer les fonctionnalités développées et testées sur la branche secondaire dans la branche de production stable.

1. Changement de contexte vers la branche principale :

Avant de procéder à l'intégration, il est nécessaire de se repositionner sur le tronc commun du projet :

- **Retour sur la branche main (`git checkout main`) :** Permet de quitter la branche de développement pour revenir sur la branche principale.

2. Intégration du code (Fusion) :

L'opération de fusion combine les historiques des deux branches :

- **Commande de fusion (`git merge dev`) :** Git rapatrie les modifications validées sur la branche dev (comme la fonction addition) vers la branche main.
- **Résultat de l'opération :** La branche main est désormais à jour avec les dernières fonctionnalités développées, tout en conservant une trace claire de l'origine des modifications dans l'historique du dépôt.

Fusion Fast-forward de dev dans main

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node
$ git checkout main
Switched to branch 'main'
```

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (ma
$ git merge dev
Updating 5e469ff..bac425a
Fast-forward
 index.js | 10 ++++++++~
 1 file changed, 9 insertions(+), 1 deletion(-)
```

V. Étape 4 - Gestion et résolution de conflits :

Cette étape simule un scénario réel de collaboration où deux développeurs modifient simultanément la même portion d'un fichier sur des branches différentes.

1. Création d'un historique divergent :

Pour provoquer le conflit, nous avons modifié le fichier index.js sur deux fronts parallèles :

- **Branche feature** : Création de cette branche pour ajouter la fonction soustraction(a, b).
- **Branche main** : Pendant ce temps, une mise à jour de la documentation (commentaires) a été effectuée directement sur la fonction addition dans la branche principale.

2. Détection du conflit (Merge Conflict) :

Lors de la tentative de fusion de la branche feature vers main (git merge feature) , Git a interrompu l'opération en signalant un **conflit de contenu** dans le fichier index.js.

- **Constat** : Git ne peut pas décider automatiquement s'il doit conserver les nouveaux commentaires de main ou la nouvelle fonction de feature car ils occupent les mêmes lignes.

3. Arbitrage et résolution manuelle :

L'ouverture du fichier révèle des marqueurs de conflit spécifiques :

- <<<<<<< **HEAD** : Indique le code présent sur la branche actuelle (main).
- ===== : Sépare les deux versions divergentes.
- >>>>>>> **feature** : Indique le code provenant de la branche entrante.

Action réalisée : Nous avons procédé à une fusion manuelle en combinant les deux apports. Le code final conserve à la fois les commentaires améliorés de la branche main et la fonction soustraction de la branche feature.

4. Finalisation de la fusion :

Une fois le fichier nettoyé des marqueurs et le code vérifié, l'opération est conclue par :

- **git add index.js** : Marquage du conflit comme résolu.
- **git commit** : Enregistrement du commit de fusion (merge commit) qui réunit définitivement les deux lignées de développement.

Commit de la fonction soustraction sur feature

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (master)
$ git checkout -b feature
Switched to a new branch 'feature'

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (feature)
$ notepad index.js

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (feature)
$ git add index.js
```

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (feature)
$ git commit -m "Ajout de la fonction soustraction"
[feature d224ff5] Ajout de la fonction soustraction
1 file changed, 7 insertions(+), 2 deletions(-)
```

Retour sur main et commit des commentaires

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (feature)
$ git checkout main
Switched to branch 'main'

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git add index.js

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git commit -m "Doc: améliorer les commentaires de addition"
On branch main
nothing to commit, working tree clean
```

Merge de feature dans main et historique git log --oneline

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git commit
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   package.json

no changes added to commit (use "git add" and/or "git commit -a")

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git add package.json
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git commit --no-edit
Aborting commit due to empty commit message.

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git commit -m "Merge branch feature into main"
[main a76289c] Merge branch feature into main
1 file changed, 2 insertions(+), 2 deletions(-)

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git log --oneline
a76289c (HEAD -> main) Merge branch feature into main
61149c5 Merge branch 'feature'
c2dc024 Doc: améliorer les commentaires de addition
d224ff5 (feature) Ajout de la fonction soustraction
bac425a (dev) Ajout de la fonction addition dans index.js
5e469ff Initialisation du projet Node (README, .gitignore, index.js)
```

VI. Étape 5 - Optimisation de l'historique via le Rebase :

Cette étape vise à utiliser une technique avancée pour maintenir un historique de projet linéaire et lisible, contrairement à la fusion classique qui crée des embranchements complexes.

1. Correction technique sur branche isolée :

Nous avons simulé une intervention urgente pour améliorer la fiabilité du code :

- **Création de la branche bugfix (git checkout -b bugfix)** : Isolation d'un correctif critique.
- **Sécurisation de la fonction addition** : Le code a été modifié pour gérer les types de données non numériques. L'utilisation de `Number(a)` et `Number.isNaN()` permet de retourner 0 au lieu d'une erreur si les paramètres ne sont pas des nombres valides.
- **Validation du correctif** : Enregistrement des modifications avec un message explicatif : *"Bugfix: addition robuste (gère les NaN)"*.

2. Application du Rebase :

Plutôt que de fusionner (merge), nous avons utilisé le mécanisme de **Rebase** :

- **Action (git rebase bugfix)** : Cette commande déplace la base de la branche main pour la positionner après le dernier commit de la branche bugfix.
- **Avantage technique** : Le rebase « rejoue » les commits les uns après les autres. Cela permet d'obtenir une ligne de temps droite (linéaire), ce qui facilite grandement la lecture de l'historique et le débogage ultérieur.

3. Comparaison Merge vs Rebase :

- **Merge** : Conserve l'historique réel des branches mais peut devenir brouillon (structure en "toile d'araignée").
- **Rebase** : Offre une vision simplifiée et propre du projet, idéale pour les flux de travail professionnels en entreprise.

Commit bugfix : addition robuste

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git checkout -b bugfix
Switched to a new branch 'bugfix'

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (bugfix)
$ notepad index.js

mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (bugfix)
$ git add index.js
```



```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (bugfix)
$ git commit -m "Bugfix: addition robuste (gère les NaN)"
[bugfix 8674555] Bugfix: addition robuste (gère les NaN)
1 file changed, 3 insertions(+), 1 deletion(-)
```

Rebase de main au-dessus de bugfix - succès

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (main)
$ git log --oneline
8674555 (HEAD -> main, bugfix) Bugfix: addition robuste (gère les NaN)
a76289c Merge branch 'feature' into main
61149c5 Merge branch 'feature'
c2dc024 Doc: améliorer les commentaires de addition
d224ff5 (feature) Ajout de la fonction soustraction
bac425a (dev) Ajout de la fonction addition dans index.js
5e469ff Initialisation du projet Node (README, .gitignore, index.js)
```

VII. Étape 6 - Application sélective de modifications (Cherry-pick) :

Cette technique avancée de Git permet de sélectionner un commit spécifique sur une branche et de l'appliquer isolément sur une autre, sans avoir à fusionner l'intégralité de l'historique.

1. Identification du commit cible :

Pour isoler la modification souhaitée, nous avons consulté l'historique compact du projet :

- **Commande (git log --oneline) :** Cette commande affiche une liste simplifiée des commits avec leurs identifiants SHA-1 abrégés.
- **Sélection :** Nous avons identifié le commit correspondant au correctif de robustesse.

2. Exécution du Cherry-pick :

L'objectif était d'importer ce correctif précis sur la branche feature :

- **Changement de branche (git checkout feature) :** Positionnement sur la branche de destination.
- **Application du commit (git cherry-pick) :** Git extrait les modifications de ce commit spécifique et les applique directement sur la branche actuelle.

3. Résultat et validation :

Comme le montre la capture d'écran ci-dessous, l'opération est réussie :

- **Preuve visuelle :** Le pointeur HEAD de la branche feature se trouve désormais sur le commit de bugfix importé.

- **Intégrité** : La branche feature bénéficie maintenant de la protection contre les valeurs NaN dans la fonction addition, tout en restant indépendante des autres modifications de la branche main ou bugfix.

Résultat final : cherry-pick appliqué sur feature

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/mon-projet-node (feature)
$ git log --oneline
217a209 (HEAD -> feature) Bugfix: addition robuste (gère les NaN)
d224ff5 Ajout de la fonction soustraction
0ac425a (dev) Ajout de la fonction addition dans index.js
5e469ff Initialisation du projet Node (README, .gitignore, index.js)
```

VIII. Conclusion :

Ce TP pratique a permis de maîtriser les commandes essentielles de Git à travers un projet Node.js concret. Voici un récapitulatif des compétences acquises :

Étape	Compétence acquise
Étape 1	Initialiser un dépôt Git et un projet Node.js
Étape 2	Créer et naviguer entre branches (git branch, git checkout).
Étape 3	Fusionner des branches avec git merge.
Étape 4	Détecter et résoudre des conflits de fusion.
Partie 5	Réorganiser l'historique avec git rebase.
Partie 6	Appliquer un commit ciblé avec git cherry-pick.